

Министерство образования и науки
федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет прикладной информатики

Отчет по дисциплине: «Web-программирование»

Практическая работа 6

Выполнила: Полтавец
Елена Андреевна

Группа: К3322

Проверила: Марченко
Елена Вадимовна

Санкт-Петербург

2025

Цель: овладеть практическими навыками и умениями реализации web-серверов и использования сокетов.

Задачи:

- Ознакомиться с теоретическими основами работы с сокетами;
- Реализовать клиентскую и серверную части приложения согласно заданиям;
- Реализовать многопользовательский чат.

Ход работы

Задание 1.

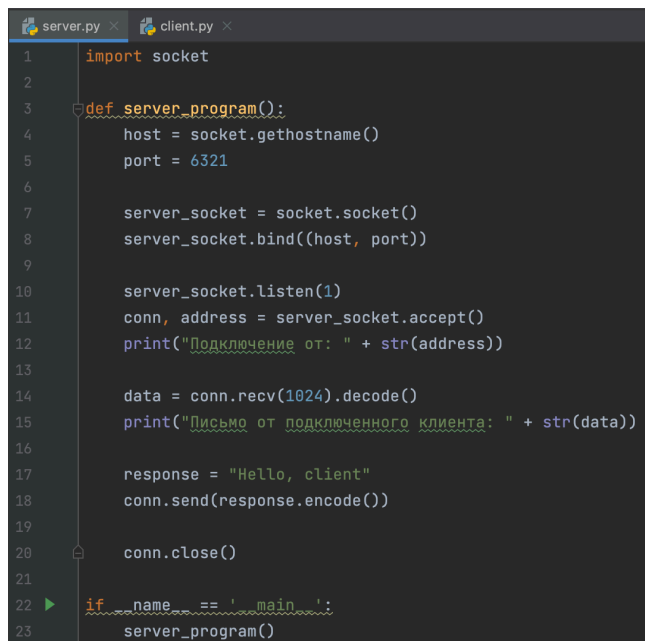
В данном задании было необходимо реализовать клиентскую и серверную части приложения. Клиент отправляет серверу сообщение «Hello, server». Сообщение должно отразиться на стороне сервера. Сервер в ответ отправляет клиенту сообщение «Hello, client». Сообщение должно отобразиться у клиента.

Серверная часть (Рисунок 1):

1. Создается сокет и связывается с хостом и портом;
2. Сервер ожидает подключения от клиента;
3. После установления соединения сервер получает сообщение от клиента и выводит его на экран;
4. Затем сервер отправляет ответ клиенту и закрывает соединение.

Клиентская часть (Рисунок 2):

1. Клиент создает сокет и подключается к серверу;
2. Клиент отправляет сообщение «Hello, server»;
3. Полученный ответ от сервера выводится на экран.

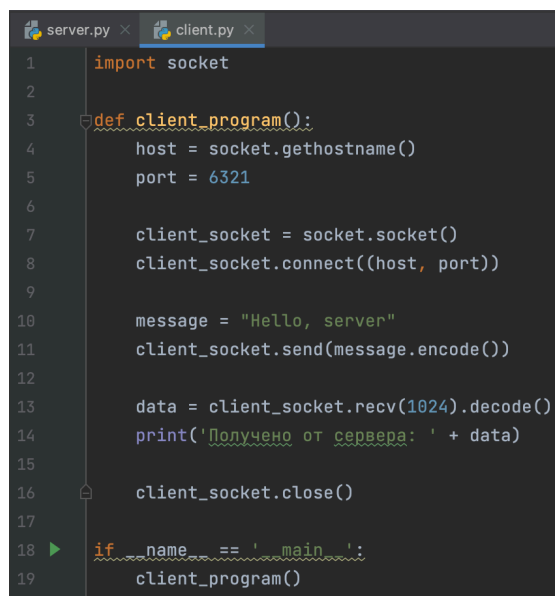


```

1  import socket
2
3  def server_program():
4      host = socket.gethostname()
5      port = 6321
6
7      server_socket = socket.socket()
8      server_socket.bind((host, port))
9
10     server_socket.listen(1)
11     conn, address = server_socket.accept()
12     print("Подключение от: " + str(address))
13
14     data = conn.recv(1024).decode()
15     print("Письмо от подключенного клиента: " + str(data))
16
17     response = "Hello, client"
18     conn.send(response.encode())
19
20     conn.close()
21
22  if __name__ == '__main__':
23     server_program()

```

Рисунок 1 – Код для сервера. Задание 1



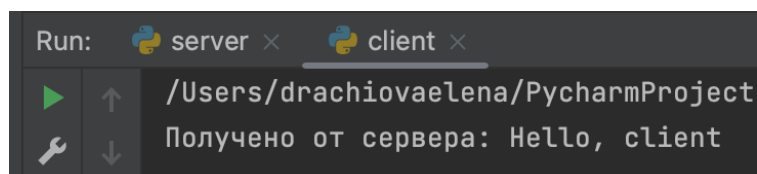
```

1  import socket
2
3  def client_program():
4      host = socket.gethostname()
5      port = 6321
6
7      client_socket = socket.socket()
8      client_socket.connect((host, port))
9
10     message = "Hello, server"
11     client_socket.send(message.encode())
12
13     data = client_socket.recv(1024).decode()
14     print('Получено от сервера: ' + data)
15
16     client_socket.close()
17
18  if __name__ == '__main__':
19     client_program()

```

Рисунок 2 – Код для клиента. Задание 1

Результат работы представлен на рисунках 3–4.



```

Run:  server  client
/Users/drachiovaelena/PycharmProject
Получено от сервера: Hello, client

```

Рисунок 3 – Получение сообщения от сервера. Задание 1

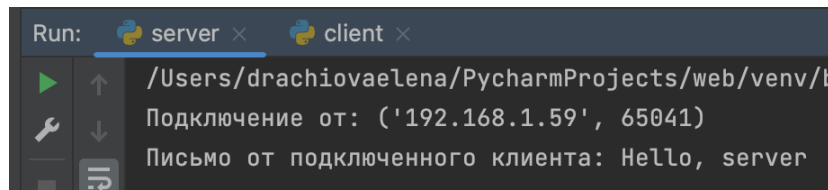


Рисунок 4 – Получение сообщения от клиента. Задание 1

Задание 2.

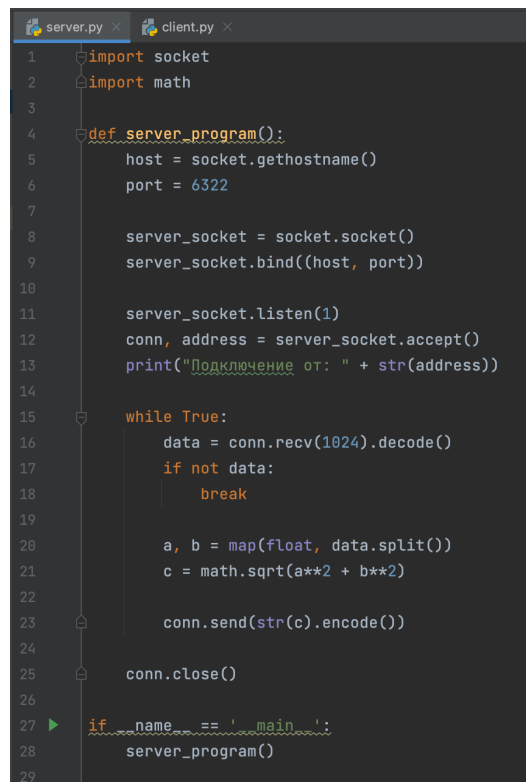
В данном задании было необходимо реализовать клиентскую и серверную части приложения. Клиент отправляет серверу сообщение «Hello, server». Сообщение должно отразиться на стороне сервера. Сервер в ответ отправляет клиенту сообщение «Hello, client». Сообщение должно отобразиться у клиента.

Серверная часть (Рисунок 5):

1. Сервер создает сокет и слушает входящие подключения на порту;
2. После подключения клиента он принимает строку, содержащую длины катетов, разделённые пробелом;
3. Сервер вычисляет гипотенузу с помощью теоремы Пифагора и отправляет результат обратно клиенту.

Клиентская часть (Рисунок 6):

1. Клиент подключается к серверу и запрашивает у пользователя длины катетов;
2. После ввода данных пользователь отправляет их серверу;
3. Клиент получает рассчитанную гипотенузу от сервера и отображает её.

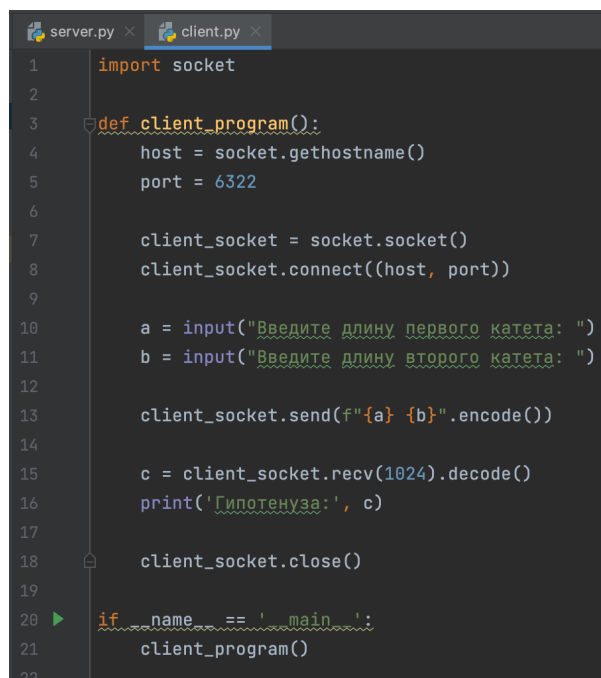


```

1  import socket
2  import math
3
4  def server_program():
5      host = socket.gethostname()
6      port = 6322
7
8      server_socket = socket.socket()
9      server_socket.bind((host, port))
10
11     server_socket.listen(1)
12     conn, address = server_socket.accept()
13     print("Подключение от: " + str(address))
14
15     while True:
16         data = conn.recv(1024).decode()
17         if not data:
18             break
19
20         a, b = map(float, data.split())
21         c = math.sqrt(a**2 + b**2)
22
23         conn.send(str(c).encode())
24
25     conn.close()
26
27 if __name__ == '__main__':
28     server_program()
29

```

Рисунок 5 – Код для сервера. Задание 2



```

1  import socket
2
3  def client_program():
4      host = socket.gethostname()
5      port = 6322
6
7      client_socket = socket.socket()
8      client_socket.connect((host, port))
9
10     a = input("Введите длину первого катета: ")
11     b = input("Введите длину второго катета: ")
12
13     client_socket.send(f"{a} {b}".encode())
14
15     c = client_socket.recv(1024).decode()
16     print('Гипотенуза: ', c)
17
18     client_socket.close()
19
20 if __name__ == '__main__':
21     client_program()
22

```

Рисунок 6 – Код для клиента. Задание 2

Результат работы представлен на рисунках 7–8.

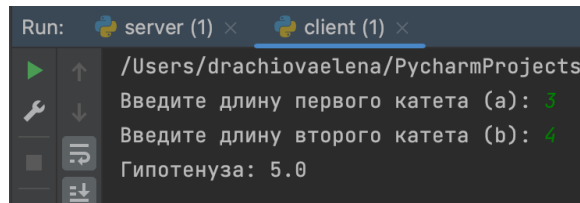


Рисунок 7 – Получение сообщения от клиента. Задание 2

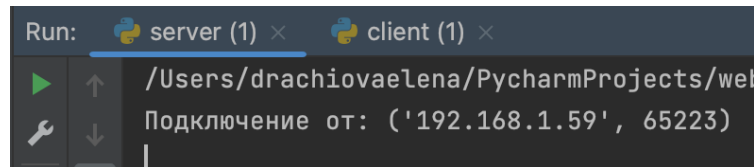


Рисунок 8 – Получение сообщения от сервера. Задание 2

Задание 3.

В данном задании было необходимо реализовать серверную часть приложения. Клиент подключается к серверу. В ответ клиент получает http-сообщение, содержащее html-страницу, которую сервер подгружает из файла index.html.

Серверная часть использует BaseHTTPRequestHandler для обработки GET-запросов. Если клиент запрашивает корневой путь (/), сервер отправляет содержимое файла index.html. Если файл не найден, сервер отправляет ответ с кодом 404 и простой HTML-страницей, сообщающей об ошибке (Рисунки 9–10).

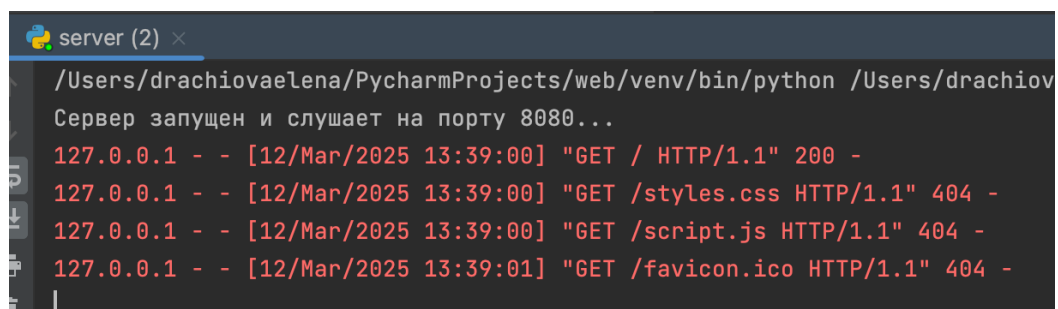
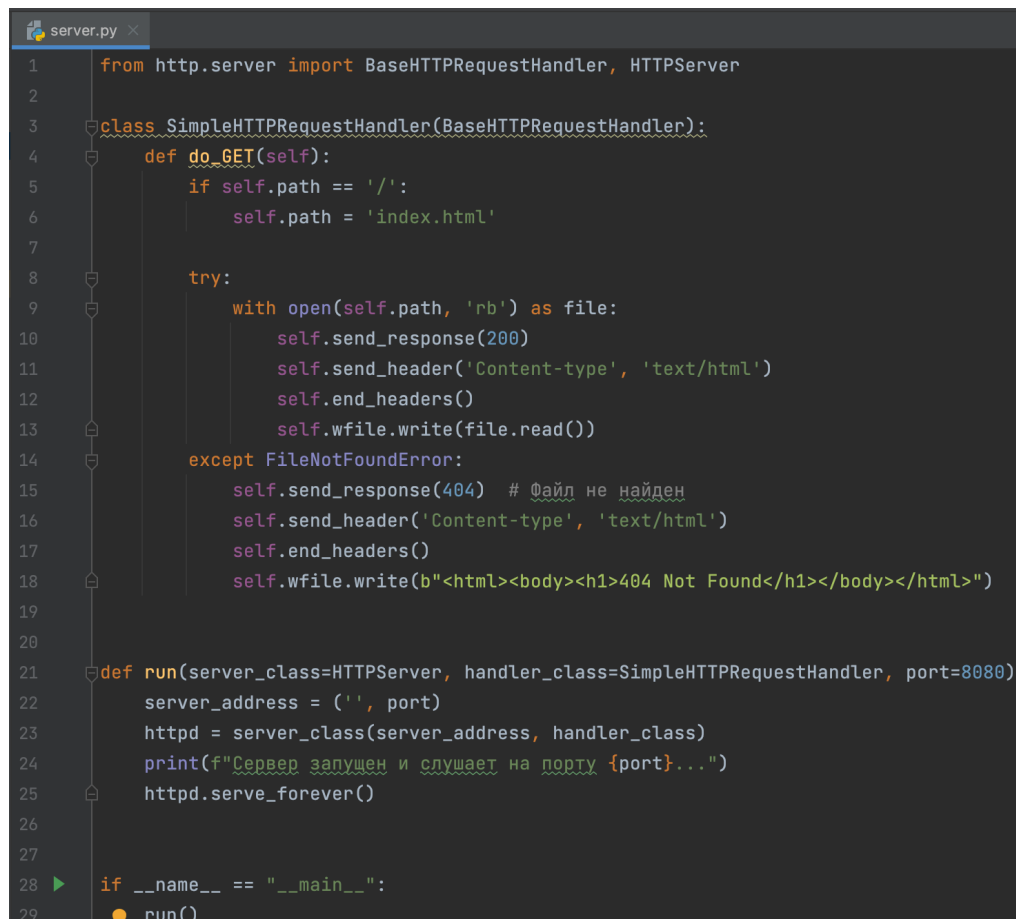


Рисунок 10 – Пример работы



```

1  from http.server import BaseHTTPRequestHandler, HTTPServer
2
3  class SimpleHTTPRequestHandler(BaseHTTPRequestHandler):
4      def do_GET(self):
5          if self.path == '/':
6              self.path = 'index.html'
7
8          try:
9              with open(self.path, 'rb') as file:
10                 self.send_response(200)
11                 self.send_header('Content-type', 'text/html')
12                 self.end_headers()
13                 self.wfile.write(file.read())
14             except FileNotFoundError:
15                 self.send_response(404) # Файл не найден
16                 self.send_header('Content-type', 'text/html')
17                 self.end_headers()
18                 self.wfile.write(b"<html><body><h1>404 Not Found</h1></body></html>")
19
20
21  def run(server_class=HTTPServer, handler_class=SimpleHTTPRequestHandler, port=8080):
22      server_address = ('', port)
23      httpd = server_class(server_address, handler_class)
24      print(f"Сервер запущен и слушает на порту {port}...")
25      httpd.serve_forever()
26
27
28  if __name__ == "__main__":
29      run()

```

Рисунок 10 – Код для сервера. Задание 3

Задание 4.

В данном задании был реализован многопользовательский чат, позволяющий пользователям обмениваться сообщениями.

Серверная часть (Рисунок 11):

1. Создание сервера. Сервер запускается на локальном адресе и прослушивает указанный порт. Он готов принимать подключения от клиентов;
2. Обработка клиентов. Когда клиент подключается, сервер создает новый поток для обработки этого клиента. Он сначала получает имя пользователя от клиента и сохраняет его вместе с сокетом в словаре;
3. Получение сообщений. Сервер принимает сообщения от подключенного клиента в цикле. Если сообщение получено, сервер добавляет имя

пользователя к сообщению и рассылает его всем остальным подключенным клиентам;

4. Удаление клиента. Если клиент отключается или возникает ошибка, сервер удаляет этого клиента из списка активных подключений и освобождает ресурсы.



```
6
7 def handle_client(client_socket):
8
9     client_name = client_socket.recv(1024).decode()
10    client_names[client_socket] = client_name
11
12    while True:
13        try:
14            message = client_socket.recv(1024).decode()
15            if message:
16                broadcast(f"{client_name}: {message}", client_socket)
17            else:
18                remove_client(client_socket)
19                break
20        except:
21            remove_client(client_socket)
22            break
23
24    def broadcast(message, client_socket):
25        for client in client_sockets:
26            if client != client_socket:
27                try:
28                    client.send(message.encode())
29                except:
30                    remove_client(client)
31
32
33    def remove_client(client_socket):
34        if client_socket in client_sockets:
35            client_sockets.remove(client_socket)
36
37
38    def main():
39        server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
40        server.bind(("0.0.0.0", 8084))
41        server.listen(5)
42        print("Сервер запущен и ждёт подключения...")
43
44        while True:
45            client_socket, address = server.accept()
46            print(f"Подключен клиент {address}")
```

Рисунок 11 – Код для сервера. Задание 4

Клиентская часть (Рисунок 12):

1. Подключение к серверу. Клиент создает сокет и выполняет подключение к серверу, указывая его адрес и порт;

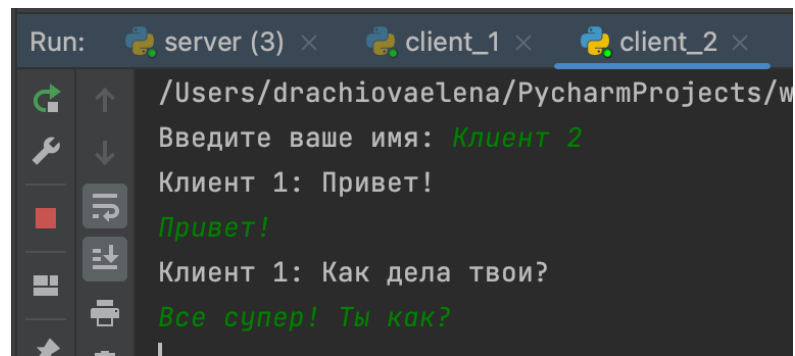
2. Запрос имени пользователя. Прежде чем отправить сообщение, клиент запрашивает у пользователя ввод имени, которое затем отправляется на сервер для идентификации;
3. Получение сообщений. Клиент запускает отдельный поток для получения сообщений от сервера, которые отображаются в консоли;
4. Отправка сообщений. Клиент может вводить сообщения в консоли. Эти сообщения отправляются на сервер и рассылаются другим подключенным клиентам. При вводе команды "exit" клиент закрывает соединение с сервером и завершает программу.



```
1 import socket
2 from threading import Thread
3 import sys
4
5
6 def receive_messages(client_socket):
7     while True:
8         try:
9             message = client_socket.recv(1024).decode()
10            print(message)
11        except:
12            print("Ошибка соединения. Вы отключены!")
13            client_socket.close()
14            break
15
16
17 def main():
18     host = socket.gethostname()
19     server_port = 8084
20     client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
21     client_socket.connect((host, server_port))
22
23     name = input("Введите ваше имя: ")
24     client_socket.send(name.encode())
25
26     Thread(target=receive_messages, args=(client_socket,)).start()
27
28     while True:
29         message = input()
30         if message.lower() == 'Выход':
31             client_socket.close()
32             sys.exit()
33         client_socket.send(message.encode())
34
35
36 if __name__ == "__main__":
37     main()
```

Рисунок 12 – Код для клиента. Задание 4

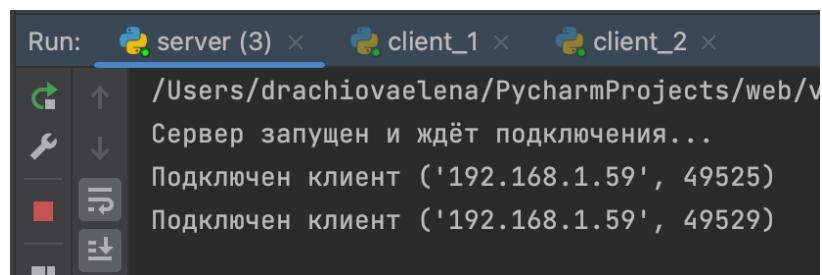
Пример работы приведен на рисунках 13–14.



The screenshot shows the Run console in PyCharm with three tabs: 'server (3)', 'client_1', and 'client_2'. The 'client_2' tab is active. The output text is as follows:

```
/Users/drachiovaelena/PycharmProjects/w
Введите ваше имя: Клиент 2
Клиент 1: Привет!
Привет!
Клиент 1: Как дела твои?
Все супер! Ты как?
```

Рисунок 13 – Пример общения клиентов. Задание 4



The screenshot shows the Run console in PyCharm with three tabs: 'server (3)', 'client_1', and 'client_2'. The 'server (3)' tab is active. The output text is as follows:

```
/Users/drachiovaelena/PycharmProjects/web/v
Сервер запущен и ждёт подключения...
Подключен клиент ('192.168.1.59', 49525)
Подключен клиент ('192.168.1.59', 49529)
```

Рисунок 14 – Пример подключения клиентов к серверу. Задание 4

Вывод: были изучены основы работы с сокетами, получены практические навыки работы, реализованы предложенные задания.